

CS 610 Coding Assignment 3

Winter 2026

Total points: 12 (include 2 extra bonus points)

Learning Outcomes

- Apply CUDA streams and concurrency to build a pre-fetch pipeline that overlaps data transfers with computation and quantitatively evaluate its performance benefits.
- Design and implement GPU kernels for data-parallel workloads, including batched matrix multiplication and pixel-wise image processing, while validating correctness against CPU implementations.
- Analyze and optimize GPU memory access patterns, including transforming data layouts (AoS $\rightarrow$ SoA) and using shared memory tiling to reduce global memory traffic.

Task 1 (4 points)

- 1) Based on the code of Task 2 of Coding Assignment 2, implement batched matrix-matrix multiplication, which performs a series of individual matrix-matrix multiplication operation with different sets of input matrices that share the same dimensions. You need to create a new source code file named: task01.cu.

- 2) For batch size = K, your program should logically does

```
For (int i = 0; i < K; i++) {  
    C[i] = A[i] · B[i]  
}
```

- 3) A[i] (input), B[i] (input), and C[i] (output) are matrices of batch i.
- 4) The input matrices should be on host memory in the beginning. A[i] and B[i] are copied to device memory before computation and output matrix C[i] is copied back the host memory after computation.

```
// pre-fetch inputs for the first batch  
H2D(A[0])  
H2D(B[0])  
For (int i = 0; i < K; i++) {  
    If (i+1 < K) { // pre-fetch inputs for next batch  
        H2D(A[i+1])  
        H2D(B[i+1])  
    }  
    Call Kernel (C[i] = A[i] · B[i])  
    D2H(C[i])  
}
```

- 5) Use CUDA concurrent execution feature (i.e., CUDA streams) to build pre-fetch pipeline that overlaps computations with memory copies
- 6) Report the time and performance of your batched matrix-matrix multiplication with and without pre-fetch pipeline when K=10 and m=k=n=10000.
- 7) Same as previous tasks, you should use your CPU matrix-matrix multiplication to verify correctness.

Task 2 (4 points)

In task 2 we are going to implement a kernel that convert a color image into a grayscale image. The image is in [PPM format](#). The functions to read color PPM and write grayscale PPM are provided in task02.cu. Complete the following changes:

- 1) Implement the “image_to_grayscale” kernel by having each thread calculate the conversion for each pixel:
$$\text{grayscale_output} = r * 0.21 + g * 0.72 + b * 0.07$$

Hint: uchar3 type contains three members: x, y, z, which correspond to r, g, b value.
- 2) 2.2 Complete the main function to handle all input/output allocation and copy. Also, invoke the “image_to_grayscale” kernel.
- 3) Timing the execution time of the kernel.
- 4) The input image array of uchar3 contains RGB data in RGBRGBRGBRGR.... (array of structures order), which is not efficient for read for GPU. Adjust the necessary parts of the code (host and device) to organize data as RRRR...GGGG...BBBB... (structure of arrays order). You can create a second kernel that takes three uchar arrays instead of one uchar3 array.
- 5) Report the changes in execution time.

Task 3 (4 points)

In task 3 we are going to implement a kernel that apply blur effect on a color image by calculating 2D convolutions. The functions to read and write color PPM are provided in task03.cu. Complete the following changes:

- 1) Implement the “image_blur_A”, “image_blur_B”, and “image_blur_C” “image_blur_D” kernels by applying convolution using three different filters.

Filter	Radius	Weights
A	1	$1/(1*2+1)^2$
B	2	$1/(2*2+1)^2$
C	4	$1/(4*2+1)^2$
D	8	$1/(8*2+1)^2$

For each filter type, you need apply the convolution on all three RGB channels separately.

$$\begin{aligned} \text{output_r}[y, x] &= \frac{1}{(\text{radius} * 2 + 1)^2} \sum_{y-\text{radius}}^{y+\text{radius}} \sum_{x-\text{radius}}^{x+\text{radius}} \text{input_r}[y, x] \\ \text{output_g}[y, x] &= \frac{1}{(\text{radius} * 2 + 1)^2} \sum_{y-\text{radius}}^{y+\text{radius}} \sum_{x-\text{radius}}^{x+\text{radius}} \text{input_g}[y, x] \\ \text{output_b}[y, x] &= \frac{1}{(\text{radius} * 2 + 1)^2} \sum_{y-\text{radius}}^{y+\text{radius}} \sum_{x-\text{radius}}^{x+\text{radius}} \text{input_b}[y, x] \end{aligned}$$

- 2) For out of bound cases, wrap the index (x, y) as follows:
 1. if (x < 0)
 - a. x += IMAGE_DIM;
 2. if (x >= IMAGE_DIM)
 - a. x -= IMAGE_DIM;
 3. if (y < 0)
 - a. y += IMAGE_DIM;
 4. if (y >= IMAGE_DIM)
 - a. y -= IMAGE_DIM;

- 3) Since neighboring threads may load the same data, apply tiling with shared memory to reduce memory load costs. Let a block of thread collaboratively load a tile of input data with halo regions (extra data near boundary), then each thread works on its on pixel.
- 4) Report execution times before and after you apply each optimization for all three kernels.